

SQL Injection Attacks on DVWA

Warning:

This environment was created in a controlled lab environment for educational purposes only.

1) General Information

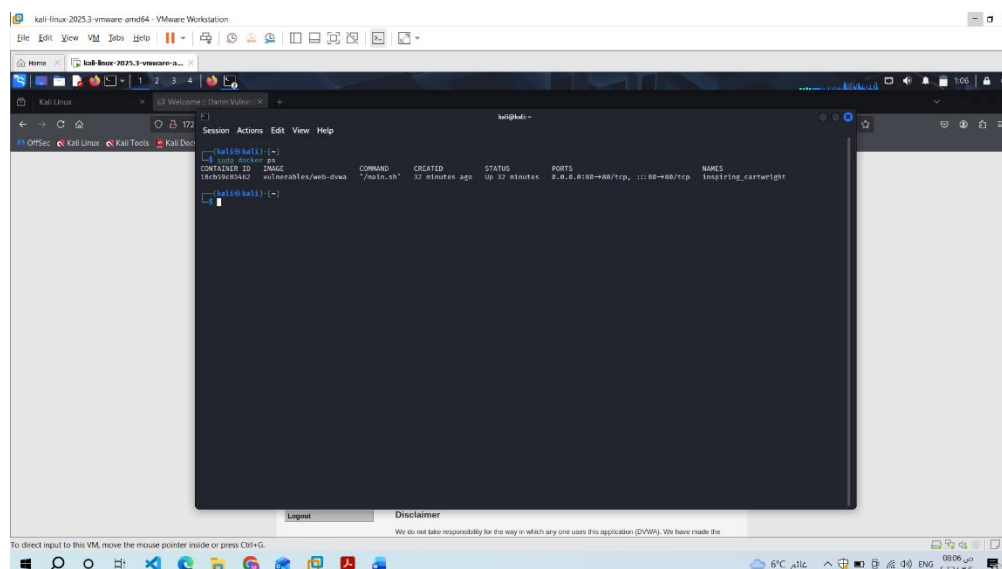
- **Assessment Date:**20/3/2026
- **Analyst Name:** Mustafa Jbili
- **Scope of Work:** Cybersecurity Analyst
- **Objective:** demonstrates how SQL Injection vulnerabilities can be exploited on a vulnerable web application using different security levels.
- ---

2) Lab's Environment :

- **Attacker Machine:** Kali Linux
- **Target Application:** Damn Vulnerable Web Application
- **Deployment Method:** Docker

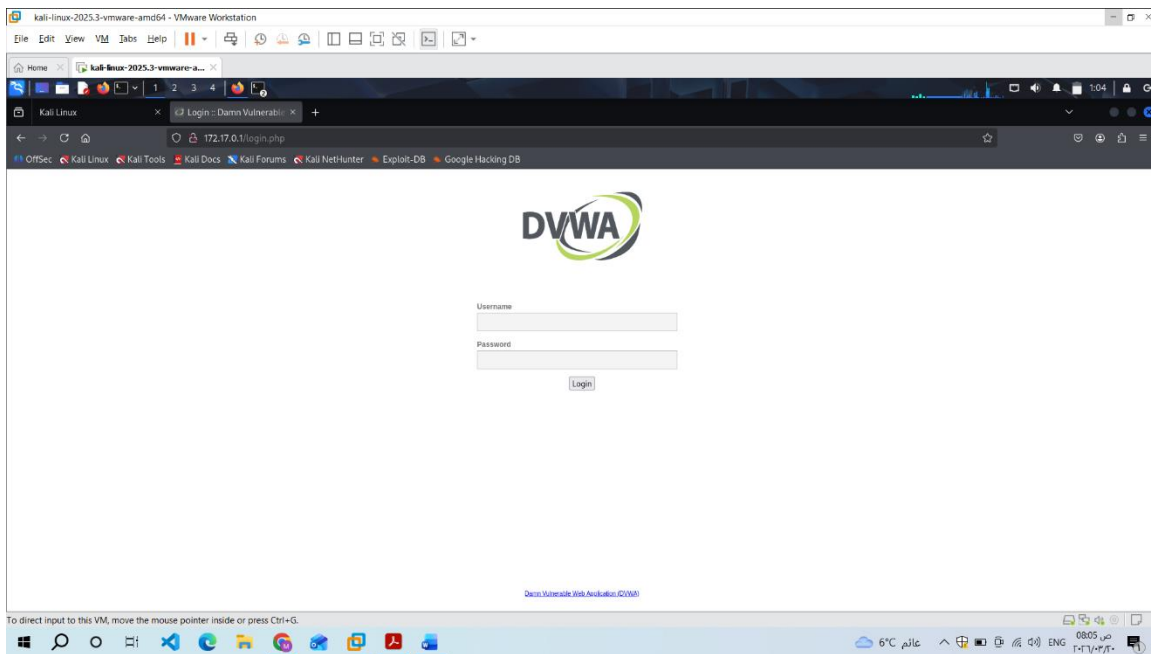
3) Accessing DVWA :

- **Run DVWA container**

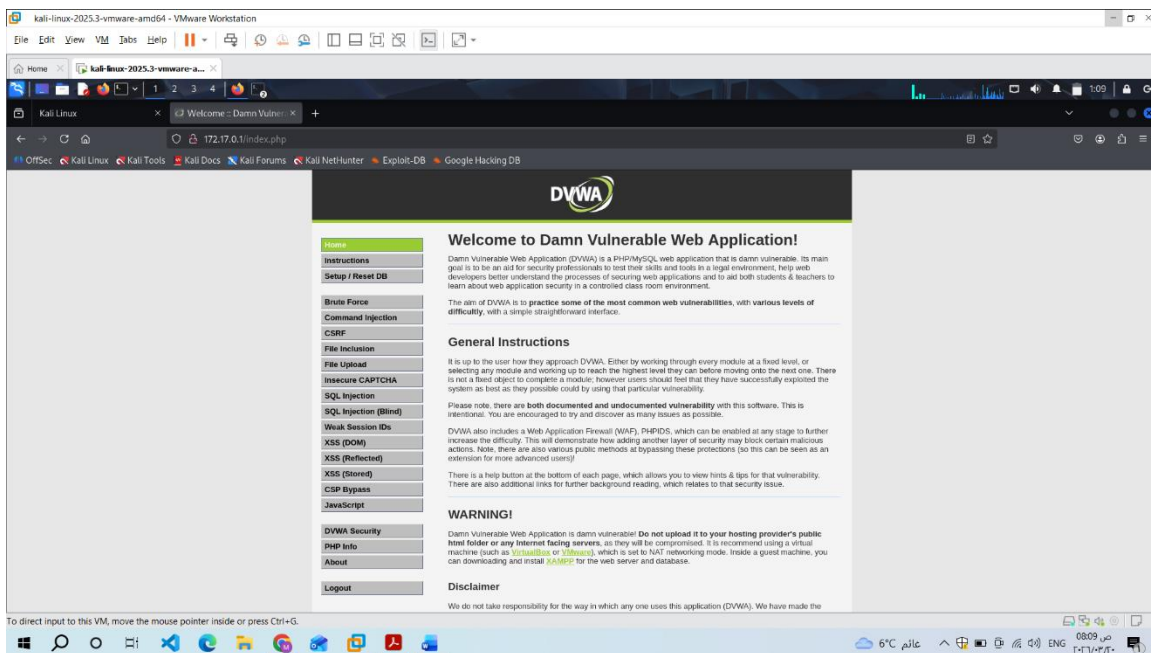


- Access the application via browser

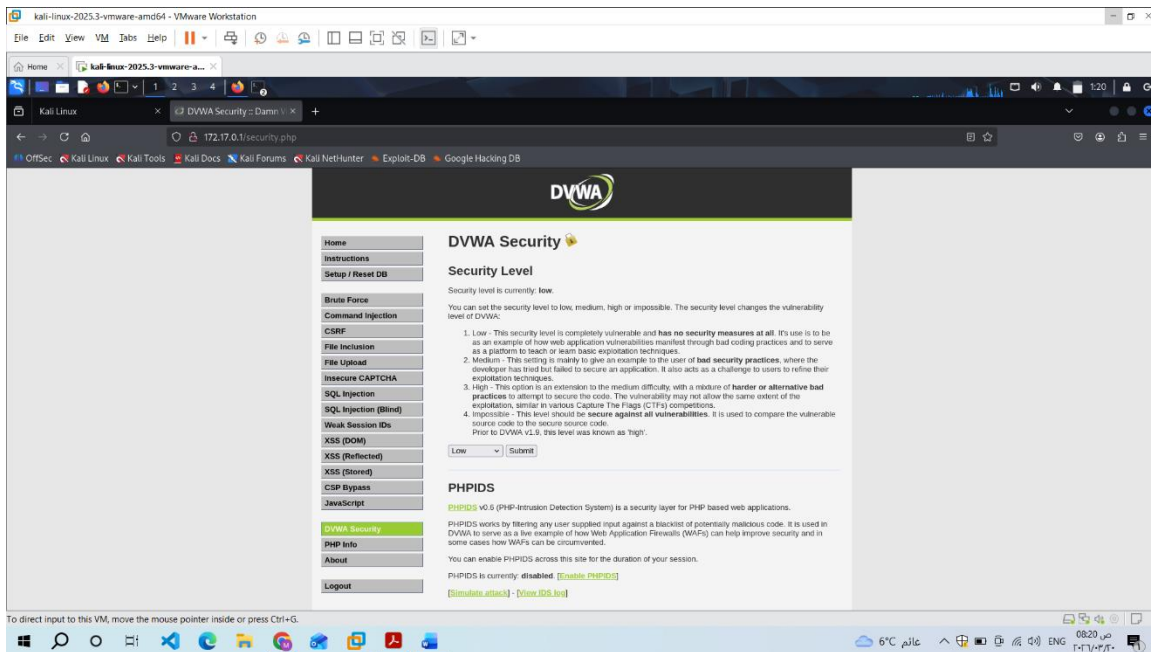
By running http://IP_ADDRESS OR <http://localhost>



- Login using credentials :



4) Select security level:

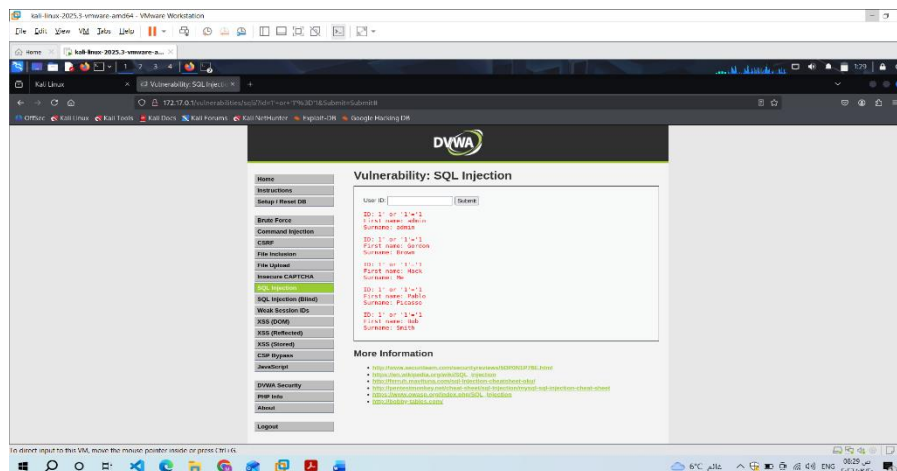


From now on we will simulate sql injection attack with each level of security :

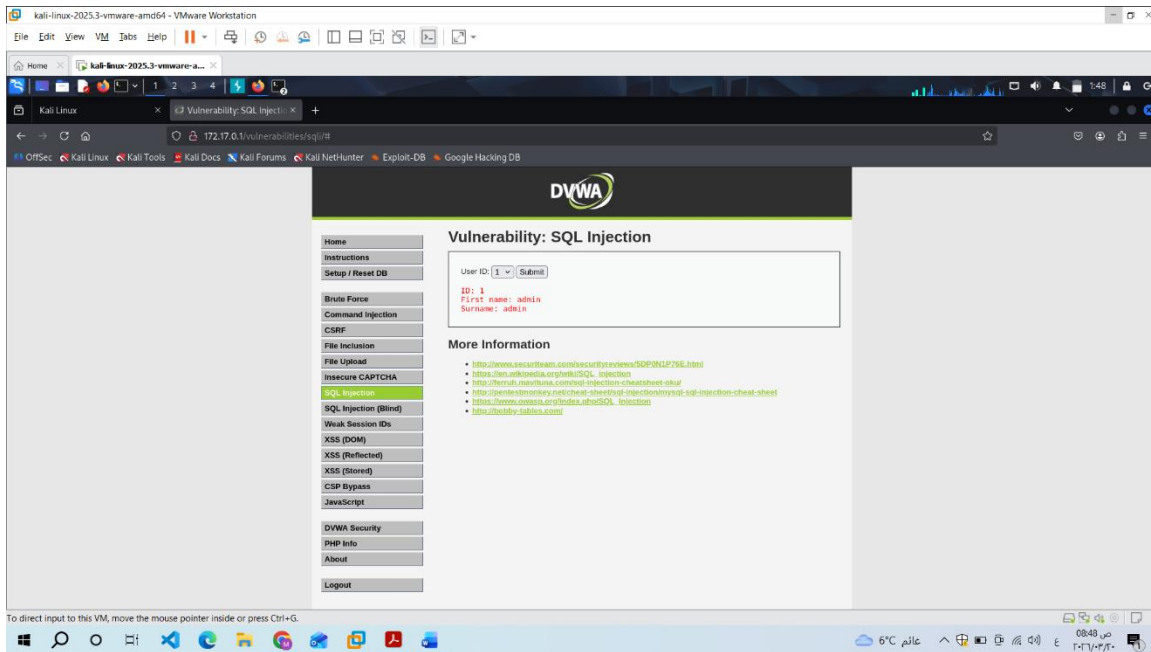
1. level low security :

By using payload : **1' OR '1'='1**

This payload modifies the SQL query to always return true, allowing access to all records in the database.



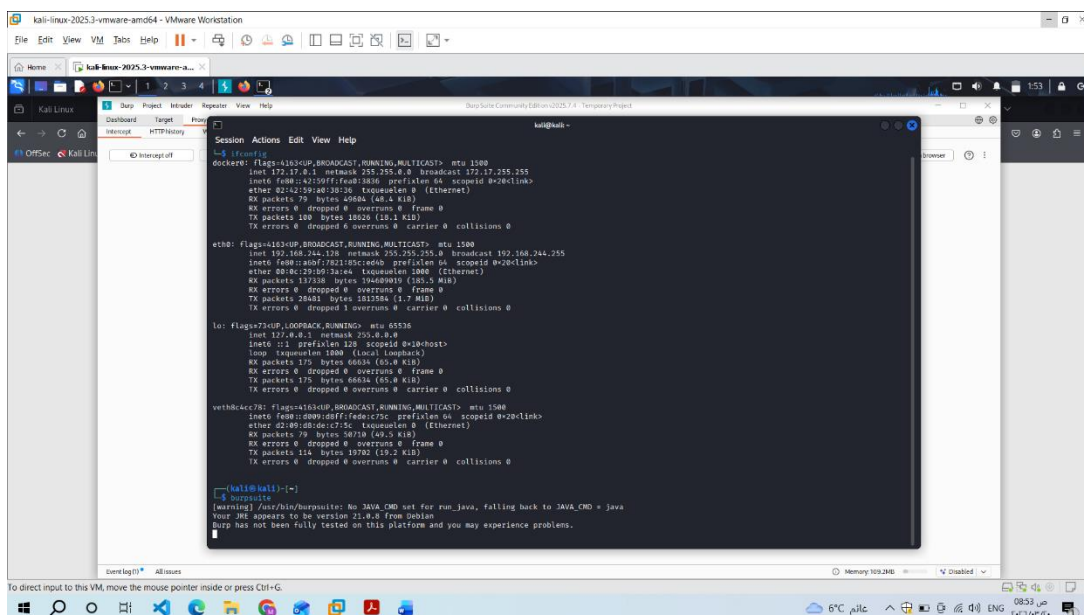
2. level medium security :



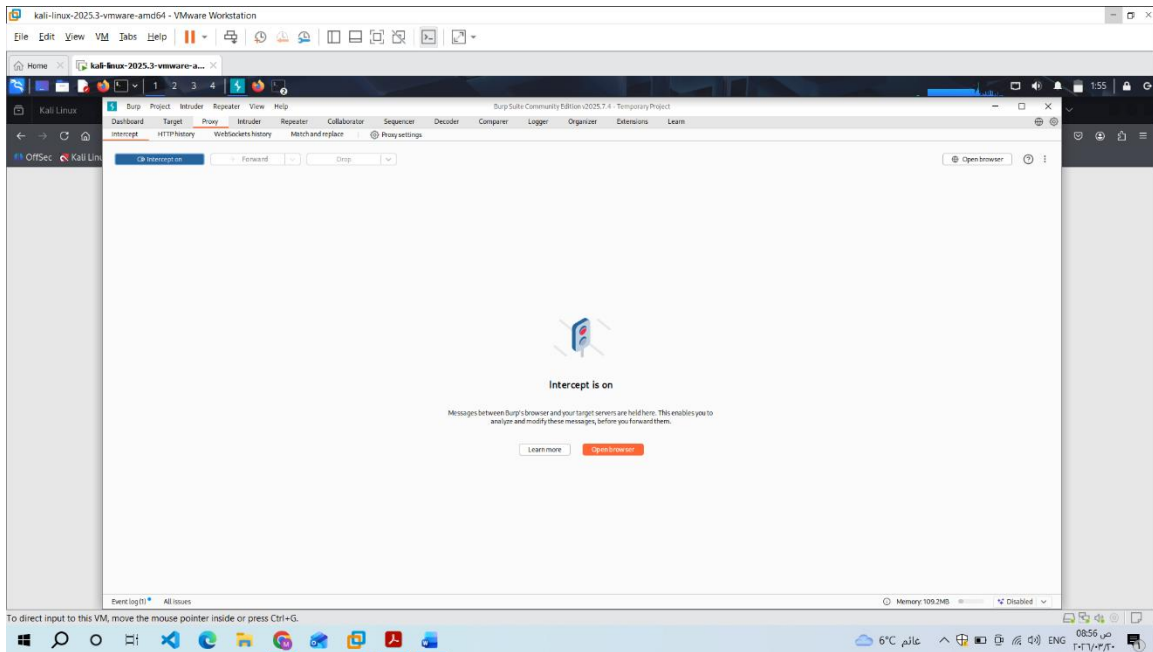
As shown , in medium level a filter has been applied and cannot be controlled text as before

For that .. we need another tactic so we can intercept the request and handle it by using **burpsuite**

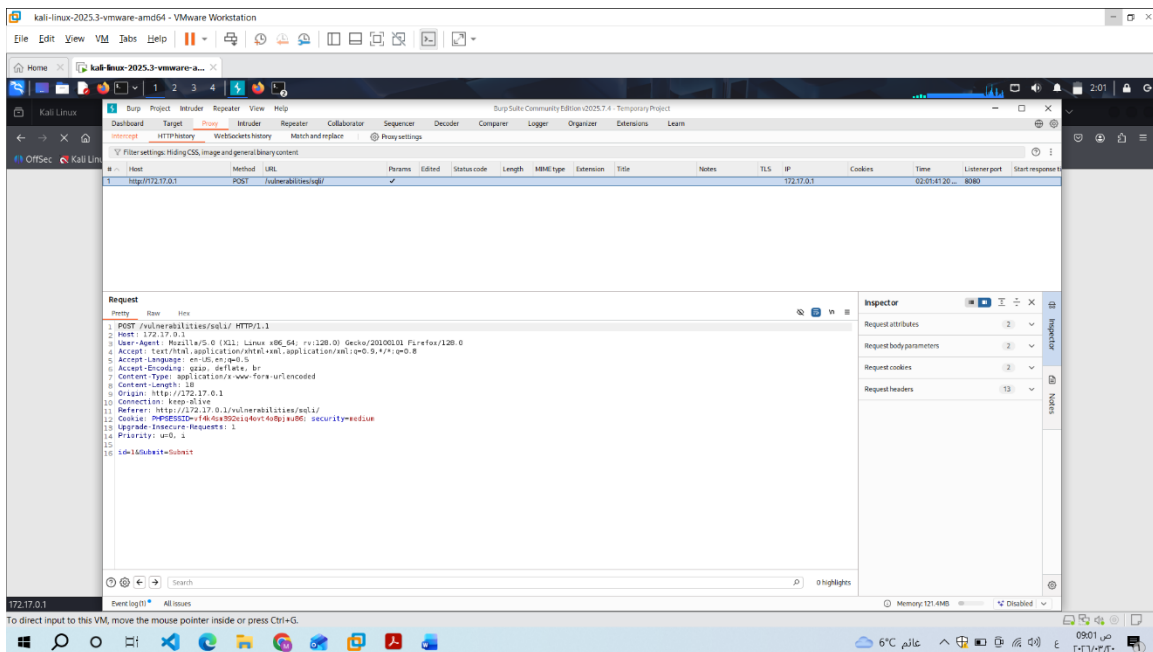
1) running burpsuite :

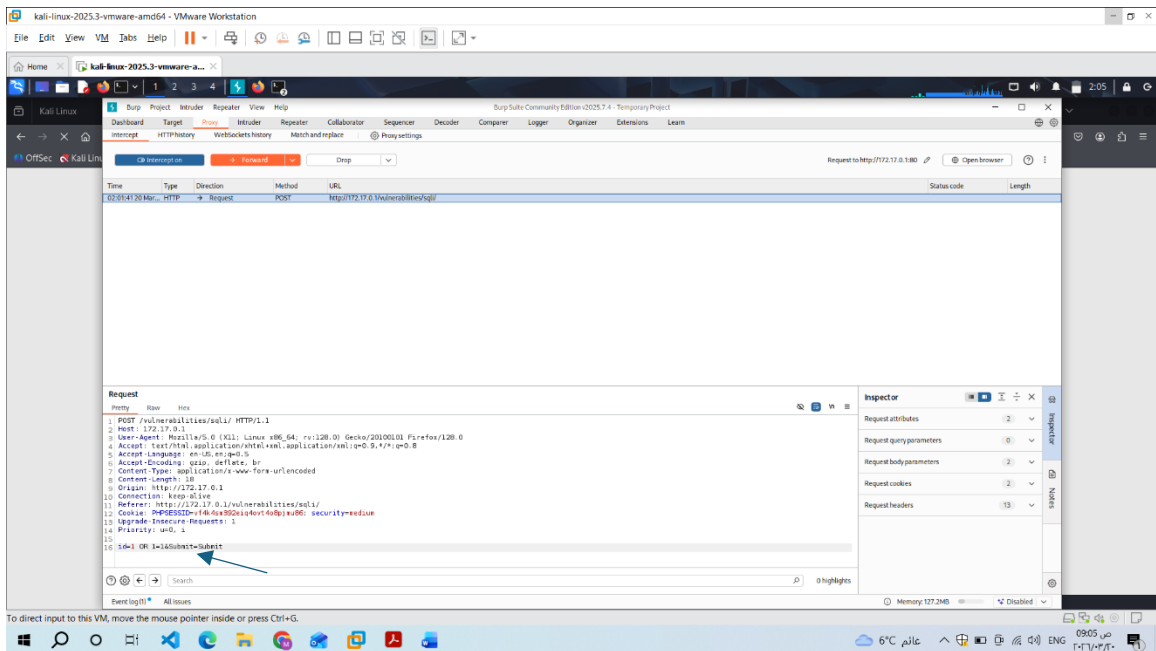


2) run Burpsuite proxy :

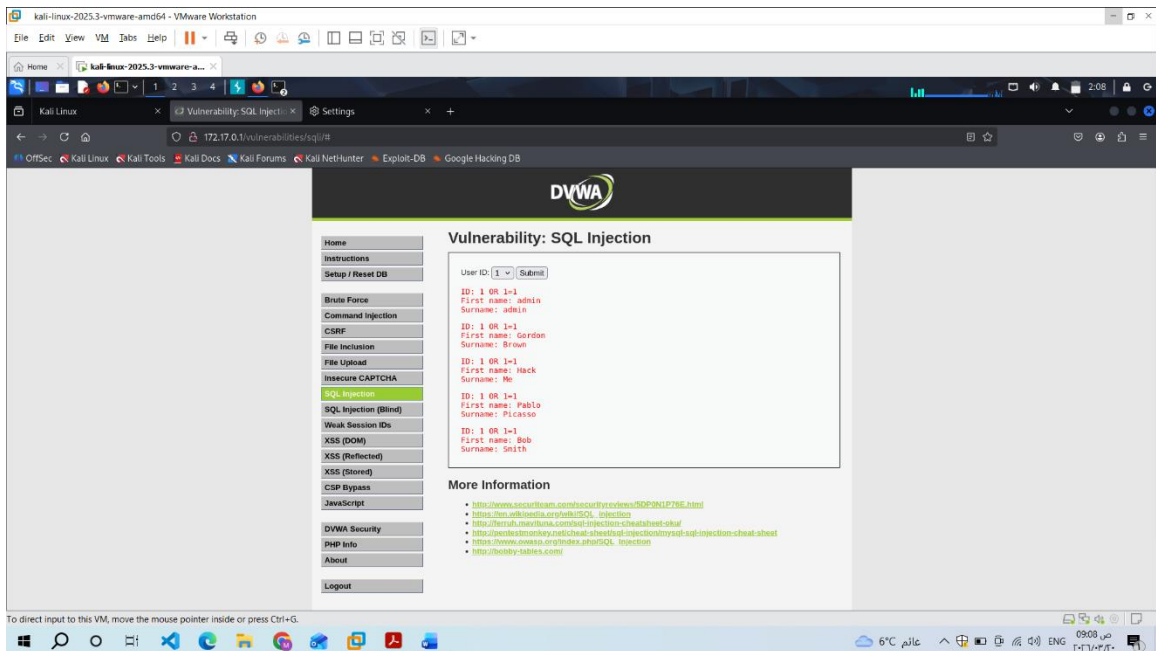


As the image below ..After submitting , the request was intercepted by the proxy and now can be modified



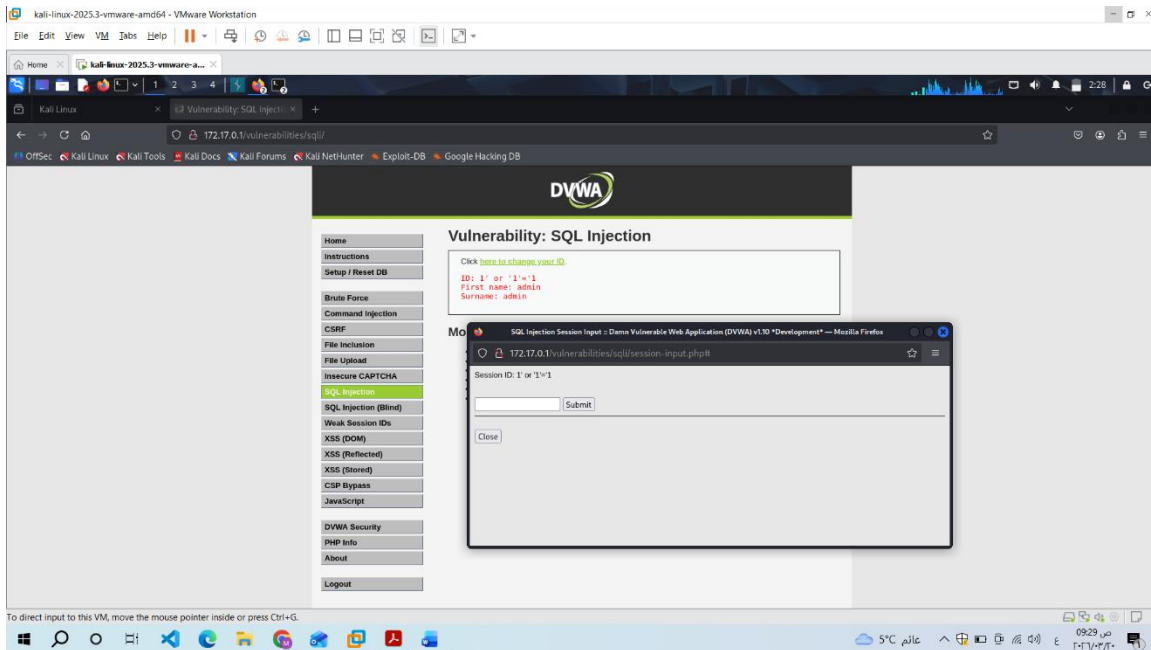


as seen it has successfully modified with payload **id=1 or 1=1** which is a true condition and all database information was displayed



2. level high security :

As image below , The level of protection has been enhanced by adding filtering so that any practice is prevented.



At this level, stronger input validation is applied. However, the attacker bypasses these protections by intercepting the request using Burp Suite and injecting a UNION-based SQL query to extract sensitive data from the database:

1' UNION SELECT user,password FROM users#

By that we are requesting the fields (username,password) from users table

